

TECNICATURA UNIVERSITARIA EN PROGRAMACION

TRABAJO PRACTICO INTEGRADOR

Catedra: Organizacion Empresarial

Prof. Gabriela Martinez (titular)

Prof. Carolina Bruno - Prof. Mario Raul Lopez - Prof. Andrea Ramos (adjuntos)

*Empresa: **TomBen** - Soluciones Informaticas*

Proceso: Solicitud y verificacion de saldo de vacaciones

Integrantes:

Tomas Rocco D Ambrosio - Comision 19

Benjamin Provera Rocha - Comision 14

https://github.com/dambro182/TPI_OE_TOMBEN

1. DESCRIPCION DE LA EMPRESA Y DEL PROCESO

1.1 LA EMPRESA

TomBen es una empresa dedicada al desarrollo de soluciones informaticas. Cuenta con un equipo de empleados distribuidos en distintas areas y actualmente busca optimizar la gestion administrativa interna mediante la automatizacion de procesos repetitivos.

1.2 PROCESO SELECCIONADO

El proceso elegido es la **solicitud de vacaciones por parte de los empleados**. Se trata de un proceso administrativo con reglas claras y criterios de decision definidos, lo que lo hace ideal para ser automatizado mediante un chatbot.

1.3 AS-IS - SITUACION ANTERIOR

Anteriormente, el empleado enviaba un correo electronico a Recursos Humanos presentando su solicitud de vacaciones. El area de RRHH debia consultar manualmente la cantidad de dias disponibles del empleado y comunicar su aprobacion o rechazo, generando demoras y posibles errores de informacion.

1.4 TO-BE - SITUACION OPTIMIZADA

Con la solucion propuesta, el empleado accede al chatbot e ingresa su numero de legajo. El sistema verifica automaticamente su identidad en la base de datos y consulta su saldo de dias disponibles. Si el saldo es suficiente, la solicitud es registrada y el balance actualizado de forma inmediata, sin intervencion humana.

2. MODELADO BPMN 2.0

2.1 DESCRIPCION DE CARRILES (LANES)

El diagrama BPMN se divide en dos carriles:

Carril	Actor	Responsabilidades
Usuario / Empleado	Empleado de TomBen	Ingresa legajo, ingresa cantidad de dias solicitados, recibe respuesta.
Sistema / Bot	Chatbot TomBen	Valida entradas, consulta base de datos, verifica saldo, aprueba o rechaza la solicitud, actualiza saldo.

Figura 1 - Carriles del proceso BPMN (lanes).

2.2 COMPUERTAS (GATEWAYS)

El proceso incluye dos compuertas de decision exclusivas (XOR):

Gateway	Pregunta	Camino SI	Camino NO
GW-1	El legajo existe en la base de datos?	Solicitar dias a tomar	Informar error y pedir reingreso
GW-2	El empleado tiene saldo suficiente?	Aprobar y registrar solicitud	Rechazar e informar saldo disponible

Figura 2 - Compuertas de decision del proceso.

2.3 DIAGRAMA BPMN 2.0

A continuacion se presenta el diagrama BPMN 2.0 del proceso de solicitud de vacaciones, representando los dos carriles (empleado y sistema/bot), los eventos de inicio y fin, las tareas de cada actor y las tres compuertas de decision exclusivas (XOR).

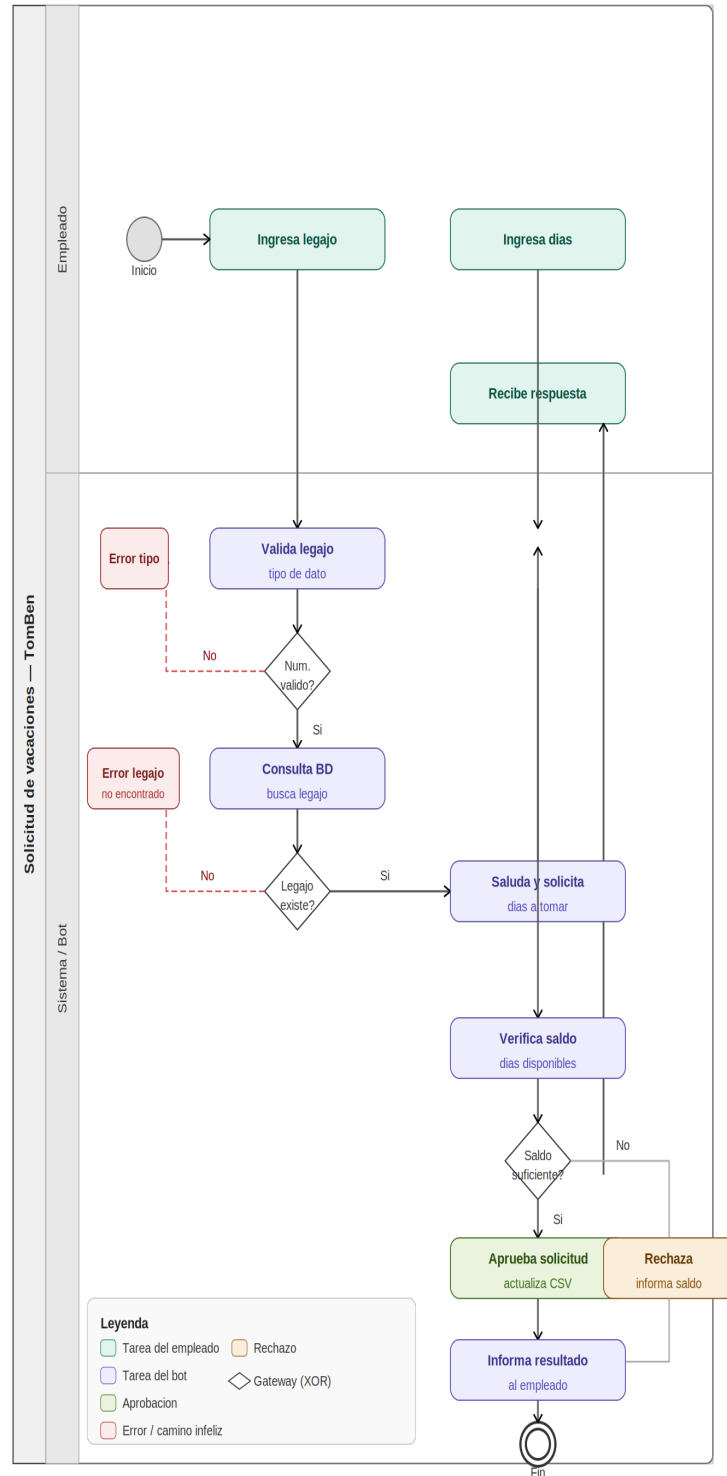


Figura 3 - Diagrama BPMN 2.0: proceso de solicitud de vacaciones TomBen.

3. ARQUITECTURA Y PROGRAMACION

3.1 STACK TECNOLÓGICO

Componente	Tecnología	Justificación
Lenguaje	Python 3	Sintaxis clara, ideal para lógica de procesos y manejo de archivos CSV.
Interfaz	Consola (CLI)	Permite simular el flujo del chatbot sin dependencias externas.
Persistencia	CSV (db.csv)	Simula una base de datos relacional simple con lectura/escritura.
IA utilizada	Claude (Anthropic)	Asistencia en diseño del flujo, corrección de código y documentación.

Figura 3 - Stack tecnológico del proyecto.

3.2 GESTIÓN DE ESTADOS (MAQUINA DE ESTADOS)

El bot implementa una máquina de estados a través de bucles while anidados. Cada bucle representa un estado de espera y solo avanza al siguiente cuando el input es válido. Los estados son:

Estado	Descripción	Transición siguiente
INICIO	El bot saluda e inicia el proceso.	-> ESPERA_LEGajo
ESPERA_LEGajo	Aguarda el ingreso del número de legajo.	-> VALIDA_LEGajo (si es número)
VALIDA_LEGajo	Verifica si el legajo existe en la BD.	-> ESPERA_DIAS (si existe)
ESPERA_DIAS	Aguarda la cantidad de días solicitados.	-> VALIDA_SALDO (si es número positivo)
VALIDA_SALDO	Compara días solicitados con saldo disponible.	-> APROBADO o RECHAZADO
APROBADO	Registra la solicitud y actualiza el CSV.	-> FIN
RECHAZADO	Informa saldo insuficiente.	-> FIN

Figura 4 - Tabla de estados de la máquina de estados del chatbot.

3.3 CAMINO INFELIZ (UNHAPPY PATH)

El sistema contempla los siguientes casos de error de entrada:

Caso	Descripción y respuesta del bot
Error 1	Input: texto en lugar de legajo (ej: 'abc') Respuesta: Error. Ingrese un número válido. -> vuelve a pedir legajo.

Error 2	Input: legajo inexistente (ej: 9999) Respuesta: Error. No se encuentra ningun empleado con ese numero de legajo. -> vuelve a pedir legajo.
Error 3	Input: texto en lugar de dias (ej: 'diez') Respuesta: Error. Ingrese un numero valido. -> vuelve a pedir dias.
Error 4	Input: 0 o numero negativo como dias Respuesta: No puede solicitar dias negativos. -> vuelve a pedir dias.
Error 5	Input: mas dias de los disponibles (ej: 21 con saldo 8) Respuesta: Saldo insuficiente. Usted solo cuenta con X dia/s disponibles. -> fin del proceso.

Figura 5 - Casos de error y respuestas del bot (camino infeliz).

4. DICCIONARIO DE DATOS

4.1 ENTIDAD: EMPLEADO (ARCHIVO DB.CSV)

Campo	Tipo	Descripción	Ejemplo
legajo	Entero	Identificador unico del empleado.	101
nombre	Texto	Nombre del empleado.	Juan
dias_disponibles	Entero	Dias de vacaciones disponibles.	15

Figura 6 - Estructura de la entidad empleado en db.csv.

4.2 VARIABLES DEL PROCESO

Variable	Tipo	Descripción	Ejemplo
legajo	Entero	Legajo ingresado por el usuario.	102
legajo_existente	Entero / None	Legajo si fue encontrado, None si no existe.	102
nombre_empleado	Texto	Nombre del empleado identificado.	Maria
dias_empleado	Entero	Saldo de dias disponibles del empleado.	8
dias_solicitados	Entero	Cantidad de dias que el usuario desea solicitar.	5
lista_empleados	Lista de dicts	Carga en memoria todos los registros del CSV.	[{...}]

Figura 7 - Variables del proceso utilizadas en main.py.

4.3 ARCHIVO DE PERSISTENCIA

Archivo	Formato	Descripción
db.csv	CSV	Base de datos de empleados. Se lee al inicio del proceso y se sobrescribe al finalizar, actualizando el saldo de dias del empleado que realizo la solicitud.

Figura 8 - Archivo de persistencia del sistema.

5. MANUAL DE USUARIO

5.1 REQUISITOS

Requisito	Detalle
-----------	---------

Python	Version 3.6 o superior.
Archivos necesarios	main.py y db.csv en la misma carpeta.
Sistema operativo	Windows, macOS o Linux.

Figura 9 - Requisitos para ejecutar el chatbot.

5.2 EJECUCION

Para iniciar el chatbot, abrir una terminal en la carpeta del proyecto y ejecutar:

```
python main.py
```

5.3 FLUJO DE USO

Paso	El bot dice...	El usuario ingresa...
1	Bienvenido al sistema de solicitud de vacaciones de TomBen. A continuacion se verificara su saldo de dias disponibles. Legajo:	Su numero de legajo (ej: 101)
2	Bienvenido Juan, a continuacion ingrese cuantos dias de vacaciones le gustaria solicitar. Dias solicitados:	Cantidad de dias (ej: 5)
3a (OK)	Verificacion exitosa. Usted cuenta con saldo suficiente para 5 dia/s. Su solicitud ha sido registrada. Dias restantes: 10. Saludos, TomBen Chatbot.	-
3b (Rechazo)	Saldo insuficiente. Usted solo cuenta con X dia/s disponibles. No es posible procesar una solicitud de Y dia/s.	-

Figura 10 - Flujo de interaccion del usuario con el chatbot.

5.4 DATOS DE PRUEBA DISPONIBLES EN DB.CSV

Legajo	Nombre	Dias disponibles
101	Juan	15
102	Maria	8
103	Paco	12
104	Tomas	5
105	Joaquin	15
106	Benjamin	21
107	Goran	12

Figura 11 - Datos de prueba cargados en la base de datos (db.csv).

6. HERRAMIENTAS DE IA UTILIZADAS

Durante el desarrollo del presente trabajo se utilizo **Claude (Anthropic)** como herramienta de asistencia. Las consultas realizadas incluyeron:

Consulta realizada	Uso en el proyecto
Diseno del diagrama de maquina de estados	Definir los estados del bot y las transiciones entre ellos.
Revision y correccion del codigo Python	Ajustar mensajes del bot para reflejar verificacion de saldo.
Redaccion del diccionario de datos	Estructurar las entidades y variables del sistema.
Redaccion del manual de usuario	Definir los pasos de uso y casos de prueba.

Figura 12 - Consultas realizadas a herramientas de IA durante el desarrollo.

Las capturas de pantalla de las consultas realizadas se adjuntan como anexo en el repositorio de GitHub.

7. CONSULTAS REALIZADAS A HERRAMIENTAS DE IA

A continuación se presentan las capturas de pantalla de las consultas realizadas a Claude (Anthropic) durante el desarrollo del presente trabajo.

Máquina de estados en un chatbot

Una máquina de estados modela la conversación como un conjunto de **estados** y **transiciones** entre ellos, según la entrada del usuario.

Concepto base

[Estado actual] + [Input del usuario] → [Acción] + [Nuevo estado]

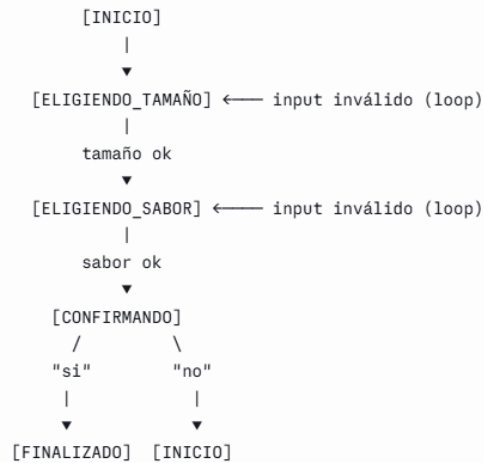
Ejemplo concreto: chatbot de pedido de pizza

Estados posibles:

INICIO → ELIGIENDO_TAMAÑO → ELIGIENDO_SABOR → CONFIRMANDO → FINALIZADO

Figura 13 - Consulta: que es una maquina de estados y como se aplica a un chatbot? (parte 1)

Diagrama de flujo



Patrones útiles para escalar

1. Tabla de transiciones (más declarativo, más fácil de mantener):

```
python
TRANSICIONES = {
    "INICIO": {"*": "ELIGIENDO_TAMAÑO"},
    "ELIGIENDO_TAMAÑO": {"chica": "ELIGIENDO_SABOR",
                        "mediana": "ELIGIENDO_SABOR",
                        "grande": "ELIGIENDO_SABOR"},
    "ELIGIENDO_SABOR": {"muzzza": "CONFIRMANDO",
                       "napolitana": "CONFIRMANDO"},
    "CONFIRMANDO": {"si": "FINALIZADO",
                   "no": "INICIO"},
}
```

Figura 14 - Consulta: maquina de estados en un chatbot (parte 2 - diagrama de flujo)

Patrones útiles para escalar

1. **Tabla de transiciones** (más declarativo, más fácil de mantener):

```
python
TRANSCIONES = {
    "INICIO": {"*": "ELIGIENDO_TAMAÑO"},
    "ELIGIENDO_TAMAÑO": {"chica": "ELIGIENDO_SABOR",
                        "mediana": "ELIGIENDO_SABOR",
                        "grande": "ELIGIENDO_SABOR"},
    "ELIGIENDO_SABOR": {"muzza": "CONFIRMANDO",
                        "napolitana": "CONFIRMANDO"},
    "CONFIRMANDO": {"si": "FINALIZADO",
                    "no": "INICIO"},
}
```

2. **Estados con contexto** — guardás datos extra en cada transición (como hicimos con `self.pedido`).

3. **Estado `FALLBACK`** — para manejar cuando el usuario dice algo inesperado en cualquier punto.

Cuándo es suficiente y cuándo no

Caso	¿Máquina de estados alcanza?
Flujos lineales (formularios, pedidos)	✅ Ideal
Menús con opciones fijas	✅ Ideal
Conversación libre / preguntas abiertas	❌ Necesitás NLP o LLM
Flujos con muchos estados anidados	⚠ Se complica, usá jerarquía

Para chatbots más complejos, la máquina de estados se combina con un **LLM** que interpreta el lenguaje natural y devuelve un intent estructurado, mientras la máquina controla el flujo de la conversación.

Figura 15 - Consulta: maquina de estados en un chatbot (parte 3 - patrones y casos de uso)

"que ventajas tiene usar un CSV como base de datos en un proyecto pequeño?"

CSV como base de datos en proyectos pequeños

Usar un CSV como base de datos tiene varias ventajas reales en contextos acotados:

Simplicidad

- No requiere instalar ni configurar ningún motor de base de datos
- Se puede leer y escribir con cualquier lenguaje (Python, JS, R, etc.) sin dependencias extra
- El esquema es visible de un vistazo

Portabilidad

- Es un archivo de texto plano: se copia, mueve o comparte fácilmente
- Funciona en cualquier sistema operativo sin adaptación
- Compatible con Excel, Google Sheets, LibreOffice para exploración manual

Transparencia

- Los datos son legibles por humanos directamente
- Ideal para debugging: abris el archivo y ves exactamente qué hay
- Fácil de versionar con Git (y los diffs son legibles)

Cero infraestructura

- No hay servidor que levantar ni mantener
- Sin usuarios, permisos ni configuración de red
- Despliegue trivial: el archivo viaja con el proyecto

Integración con herramientas de análisis

- Pandas, R, Excel y casi toda herramienta de datos lo consume nativamente



Figura 16 - Consulta: que ventajas tiene usar un CSV como base de datos en un proyecto pequeno? (parte 1)

Transparencia

- Los datos son legibles por humanos directamente
- Ideal para debugging: abris el archivo y ves exactamente qué hay
- Fácil de versionar con Git (y los diffs son legibles)

Cero infraestructura

- No hay servidor que levantar ni mantener
- Sin usuarios, permisos ni configuración de red
- Despliegue trivial: el archivo viaja con el proyecto

Integración con herramientas de análisis

- Pandas, R, Excel y casi toda herramienta de datos lo consume nativamente
- Perfecto para scripts de ETL simples o reportes puntuales

¿Cuándo tiene sentido?

- Datos que no superan los ~50k-100k registros
- Acceso mayormente de lectura (o escrituras poco frecuentes)
- Un solo proceso accediendo a la vez (sin concurrencia)
- Proyectos de análisis, prototipos o scripts internos

Cuándo ya no alcanza: múltiples escrituras simultáneas, relaciones complejas entre entidades, necesidad de consultas eficientes, o datos sensibles que requieren control de acceso → ahí conviene migrar a SQLite como mínimo.

🔍 🔍 🔍 🔍 🔍

Figura 17 - Consulta: ventajas del CSV como base de datos (parte 2)